

Padrões de Codificação e Boas Práticas - Deu Branco

0. Como aplicar estas regras

Estas regras devem ser seguidas em todo código desenvolvido para o projeto. Em partes já existentes, sempre que houver manutenção ou evolução, o trecho alterado deve ser atualizado para ficar alinhado a este padrão.

1. Padrão de notação e formatação

Para todo mundo conseguir ler o código da mesma forma, vamos seguir este padrão:

Formatação

- Identação fixa (por exemplo, 4 espaços) em todos os arquivos.
- Um comando por linha, evitando linhas muito longas.
- Sempre usar espaços em torno de operadores (ex: `a + b`, não `a+b`).
- Usar espaço após vírgulas, como em `func(a, b, c)`.

Nomes

- **Classes e Interfaces:** `PascalCase` (ex.: `QuestaoController`, `JogadorService`).
(Nota: Adaptado para o padrão oficial das linguagens Java e TypeScript utilizadas no Deu Branco, sobrepondo a recomendação original de `lowerCamelCase`).
- **Variáveis e métodos:** `lowerCamelCase` (ex.: `buscarPorEmail`, `atualizarJogadorAutenticado`).
- Evitar nomes genéricos como `x`, `y`, `temp` para elementos de regra de negócio.
- Usar nomes que expressam claramente o propósito (ex.: `alternativaCorreta`, `dadosAtualizados`).

Estruturas de controle

- Sempre usar chaves `{}` em `if`, `else`, `for`, `while`, mesmo com uma linha só, para evitar erros de manutenção.
-

2. Comentários e documentação mínimos

A ideia é que qualquer pessoa da equipe (Giovane, João ou Caio) consiga entender o código sem dificuldade excessiva.

- **Documentação de funções:** Funções públicas ou mais importantes devem ter uma breve documentação indicando: Propósito da função; Parâmetros principais; O que ela retorna (quando houver).

- **Comentários no meio do código:** Comentar o porquê de algo ser feito, não apenas o que está sendo feito.
 - Se for necessário comentar demais um trecho para que ele seja entendido, avaliar e extrair a lógica para um método com nome claro.
 - **Manutenção:** Sempre que o código for alterado, atualizar os comentários relacionados. Comentários desatualizados são piores do que a ausência de comentários.
-

3. Nomes significativos e Princípios SOLID

Essas regras ajudam a evitar que a regra de negócio fique dispersa e difícil de entender, garantindo a escalabilidade da aplicação.

- **Classes e Métodos:** Usar substantivos para classes (ex.: `Questao`, `AuthService`) e verbos para métodos (ex.: `criarJogador`, `validarAlternativas`).
 - **Alta Coesão e Responsabilidade Única (SRP):** Cada classe ou componente deve ter um papel principal bem definido e focado. (ex.: `JogadorController` lida estritamente com requisições HTTP e respostas; `JogadorService` lida isoladamente com as regras de negócio de contas).
 - **Garantia de Extensão Futura (Open/Closed Principle):** Seguiremos as boas práticas do SOLID para garantir que o código seja aberto para extensão, mas fechado para modificação. Se uma nova regra ou funcionalidade precisar ser adicionada, devemos estender o comportamento criando novos códigos (novos serviços, hooks, componentes React ou implementações de interfaces) em vez de alterar destrutivamente o código que já funciona.
-

4. Funções simples e sem repetição desnecessária (Clean Code)

O objetivo é evitar complexidade desnecessária e facilitar a leitura e manutenção.

- Preferir soluções simples em vez de soluções excessivamente complexas.
 - Evitar copiar e colar a mesma lógica em vários lugares; quando isso acontecer, extrair para um método reutilizável (como os métodos validadores do `QuestaoService`).
 - Evitar condicionais muito aninhadas (ifs dentro de ifs).
 - Quando possível, usar retornos antecipados (*early return*) ou quebrar a lógica em métodos auxiliares.
-

5. Testes e tratamento de erros

Esta regra existe para garantir o mínimo de segurança sem tornar o processo pesado demais.

Testes

- Funcionalidades importantes e essenciais para o sistema devem ter pelo menos um teste automatizado (unitário ou de integração).
- Antes de integrar mudanças na branch principal, executar os testes existentes (como `JogadorServiceTest`).

Erros

- Preferir lançar exceções com mensagens claras (ex: `EntityNotFoundException`) em vez de códigos numéricos sem significado.
 - Validar entradas críticas, como valores nulos, formatos inválidos ou dados fora de faixa esperada (ex: garantir que a alternativa correta seja apenas de A a E).
 - Garantir que mensagens de erro retornadas aos jogadores ou à interface frontend sejam compreensíveis.
-

6. Regra de ouro

Em caso de dúvida, priorizar código simples, legível e consistente com o padrão já adotado no projeto Deu Branco.